

Introducción

Diccionario de datos

Un **diccionario de datos (DD)** é un repositorio de información que almacena metadatos:

- Inclúe información sobre os propios datos (significado, tipos de datos, formatos, tamaños, restricións... pero pode tamén incorporar outros elementos como os programas que os utilizan ou usuarios que poden acceder a eles.
- O DD xestiona a información de forma centralizada (será consultado e / ou modificado por usuarios cos permisos necesarios), o que pode mellorar a autenticidade da información e a comunicación entre os usuarios. Un DD permite:
 - Documentar o modelo: requisitos, especificacións, deseño, ...
 - Xenerar informes sobre calquera elemento do modelo.
 - Analizar o impacto dun cambio no modelo de datos.
 - Xenerar automaticamente a estrutura da base de datos (ex: sentencias `SQL create table`).
 - Xenerar automaticamente a definición de datos en linguaxes de programación que accedan a eses datos (ex: clases Java).

Catálogo do sistema

O **catálogo do sistema** dun SXBD relacional é un conxunto de táboas que almacenan metadatos sobre a base de datos, e vistas que expoñen estes datos de forma máis amigable.

É frecuente denominalas (de forma informal) como táboas e vistas do sistema, para diferencialas das táboas e vistas "dos usuarios".

O catálogo é creado polo SXBD e mantido por el, pero é de só lectura para os usuarios.

Os usuarios poden "provocar" modificacións no catálogo, por exemplo mediante operacións DDL (ex: creando unha táboa).

O propio SXBD utiliza continuamente o catálogo para realizar as súas funcións.

O catálogo é crítico para o funcionamineto do sistema.

O catálogo do sistema

Exemplos iniciais

O **catálogo** dun SXBD relacional pode ser utilizado tanto por un usuario como polo propio SXBD.

Consultas de usuario

Como podo saber as columnas da táboa **EMP**? (Consulta aproximada usando o catálogo estándar de SQL)

```
select column_name, data_type
  from information_schema.columns
 where table_name='EMP';
```

Uso por parte do sistema

Que pasa cando executo a seguinte consulta=

```
select *
  from emp
 where sal + comm > 2000
    and deptno=10;
```

- O SXBD comproba que **emp** exista e sexa unha táboa ou vista.
- Comproba que o usuario ten permiso de **select** sobre **emp**
- Expande o ***** á lista de columnas da táboa.
- Comproba que as columnas no **where** existen e son do tipo axeitado.
- No proceso de optimización: Comprueba se existen índices, identifica os espazos físicos de almacenamento, ...
- Etc.

Para obter toda esta información, o SXBD consulta o catálogo do sistema.

Contido do catálogo

O catálogo dun SXBD relacional almacena datos dos esquemas conceptual, externo e físico (según define o estándar ANSI/X3/SPARC).

Entre outros elementos, contén:

- Definición das táboas e as súas columnas.
- As restricións de integridade (claves primarias, foráneas, ...)
- Definición das vistas.
- Descrición dos espazos de almacenamento (ficheiros, espazos para táboas, ...).

- Índices asociados ás táboas.
- Información sobre usuarios, roles e privilexios.
- Descrición das estruturas lóxicas e físicas da BD.

Ademáis, o catálogo:

- Autocontén os metadatos sobre a propia estrutura do catálogo.
- *Non contén* información externa á base de datos, como a definición (significado) dos datos, para que se usan, ou os programas que os manipulan.

O catálogo SQL estándar

Estructura

Según o estándar SQL, o catálogo sigue a seguinte estrutura:

1. **Esquema:** É unha colección de elementos individuais como táboas, vistas, triggers etc. Un esquema pertence a un só usuario (en xeral, a un *ID de autorización*). Un usuario ou ID de autorización pode ser o propietario de varios esquemas.
2. **Catálogo:** É un conxunto de esquemas (os nomes de esquema deben ser únicos nun catálogo). Nomenclatura única dos elementos da BD: `nome_catologo.nome_esquema.nome_elemento`. Todo catálogo debe conter un esquema de nome `INFORMATION_SCHEMA`, que contén a información sobre todos os esquemas (e os elementos que inclúen) nese catálogo.
3. **Cluster:** É unha colección de catálogos: Cada usuario ten asociado un cluster, que representa o conxunto de catálogos ós que ten acceso. Un cluster é o máximo ámbito sobre o que se pode executar unha consulta SQL, polo que de algún xeito será "a base de datos" tal como a ve o usuario.
4. **Entorno SQL:** É o contexto no que os datos poden existir e no que se poden realizar operacións sobre eles. Na práctica podemos consideralo unha instancia do SXBD que se está executando nunha instalación concreta.

Almacenamiento e exposición de metadatos.

O SXBD debe almacenar os metadatos nun contexto de táboas base denominado `DEFINITION_SCHEMA`, que non serán accesibles polo SQL dos usuarios. Estas "táboas" son hipotéticas, xa que o estándar deixa esta parte aberta a diferentes implementacións.

Os datos expóñense a través dunha colección de vistas no `INFORMATION_SCHEMA`.

- `SCHEMATA`: Esquemas propiedade do usuario actual (o que executa a consulta sobre esta vista).

- **TABLES** : Táboas persistentes accesibles para o usuario actual.
- **COLUMNS** : Columnas das táboas persistentes accesibles para o usuario actual.
- **VIEWS** : Vistas accesibles para o usuario actual.
- **DOMAINS** : Dominios accesibles para o usuario actual.

INFORMATION_SCHEMA é autodescriptivo: Contén información sobre as súas propias vistas en **INFORMATION_SCHEMA.VIEWS**.

Os usuarios poden *consultar* o catálogo utilizando SQL, pero *non o poden modificar* directamente. As modificacións no catálogo son habitualmente consecuencia de operacións DDL.

Casos de uso

Oracle

Oracle non segue o estándar.

A estrutura dun entorno ou instalación de Oracle ten os seguintes elementos:

- Ata a versión 11, unha única base de datos. A partir da versión 12, instalando como *Container Databases*, CDBm pode ter varias.
- Aínda que hai usuarios comúns, o habitual é que cada base de datos teña os seus usuarios propios.
- Cada usuario é o propietario dun único esquema (que ten o mesmo nome).
- Dentro dun esquema hai elementos de distintos tipos. Oracle refírese a estes elementos de forma global como obxectos (**objects**),
 - Táboas, vistas.
 - Secuencias, índices.
 - Sinónimos
 - Paquetes, procedementos, funcións, disparadores
 - ...

O nome único dun elemento (dentro da base de datos á que un usuario se conecta) será o nome do usuario (ou esquema) e o nome do obxecto unidos por un punto: **SCOTT.EMP**, por exemplo.

Oracle **Multitenant* (a partir da versión 12):

- Nunha mesma instalación podemos *enchufar* PDBs (Pluggable Databases).
- Desde o punto de vista do usuario/aplicación, unha PDB é moi similar a unha instalación dunha DB Oracle 11.

Para cada tipo de obxecto (en plural), en xeral, existen vistas cos seguintes prefixos:

- `USER_<tipo obxecto>`: Obxectos creados polo usuario.
- `ALL_<tipo obxecto>`: Obxectos ós que o usuario ten acceso (permisos).
- `DBA_<tipo obxecto>`: Todos os obxectos.
- `CDB_`: Todos os obxectos (de todas as PDBs se se consultan desde o container `CDB$R00T`). Só en Oracle 12 ou superior, con instalación como CDB. Tamén require privilexios de DBA.

Os tipos de obxecto que Oracle manexa son, entre outros:

- `OBJECTS` (Todos os tipos de obxecto)
- `TABLES`
- `VIEWS`
- `INDEXES`
- `CONSTRAINTS`
- `SYNONYMS`

Exemplos de vistas do catálogo:

- `DICTIONARY`: Describe brevemente todas as vistas do catálogo.
- `DICT`: Sinónimo de `DICTIONARY`.
- `USER_OBJECTS`: Todos os obxectos creados polo usuario.
- `ALL_VIEWS`: Vistas accesibles ó usuario.
- `DBA_INDEXES`: Todos os índices (só para DBAs).

Para a consulta dos elementos creados polo propio usuario son interesantes:

- `OBJ` (sinónimo de `USER_OBJECTS`).
- `CAT` (`USER_CATALOG`): Táboas, vistas e secuencias do usuario.
- `TABS` (`USER_TAB_COLUMNS`): Columnas das táboas do usuario.
- `COLS` (`USER_TAB_COLUMNS`): Columnas das táboas do usuario.
- `TAB`: Non confundir con `TABS`. Táboas e vistas do usuario.
- `USER_CONSTRAINTS`: Restricións (das táboas) do usuario.
- `USER_CONS_COLUMNS`: Columnas implicadas nas restricións (das táboas) do usuario.

PostgreSQL

PostgreSQL sigue o estándar en certa forma, aínda que a nomenclatura varía.

SQL Estándar	PostgreSQL
Esquema	Schema (Esquema)
Catálogo	Database (Base de datos)
Cluster	–
Entorno SQL	Cluster (Instalación)

- Un usuario pode crear múltiples bases de datos, e unha base de datos é propiedade dun só usuario.
- Un usuario pode acceder a bases de datos creadas por outros usuarios, pero non existe un nome para ese conxunto de bases de datos ás que pode acceder (o *cluster* do SQL estándar).
- Dentro dunha base de datos pode haber varios *Esquemas*. Un usuario pode crear varios esquemas nunha base de datos, e un esquema é dun só usuario.
- Cada base de datos contén un esquema de nome `INFORMATIO_SCHEMA` (ademáis do propio `pg_catalog`).

Exemplo de creación de esqmeas e táboas

```
create schema abd;
create table abd.estudiante(
    dni varchar(9),
    apellidos varchar(80) not null,
    nome varchar(40) not null,
    login varchar(50) not null,
    constraint pk_estudiante primary key(dni),
    constraint u_login_est unique(login)
);
```

```
create schema mai;
create table mai.estudiante(
    dni varchar(9),
    apellidos varchar(80) not null,
    nome varchar(40) not null,
    login varchar(50) not null,
    constraint pk_estudiante primary key(dni),
    constraint u_login_est unique(login)
);
```

```
create schema bda;
create table bda.estudiante(
```

```

dni varchar(9),
apelidos varchar(80) not null,
nome varchar(40) not null,
login varchar(50) not null,
constraint pk_estudiante primary key(dni),
constraint u_login_est unique(login)
constraint ch_mencion check (mencion in ('ES', 'SI')
);

```

Consulta utilizando o estándar INFORMATION_SCHEMA e o propio de PostgreSQL `pg_catalog`.

```

docencia => select table_schema, table_name, table_type
             from information_schema.tables
             where table_schema in ('abd', 'bda', 'mai');

```

table_squema	table_name	table_type
abd	estudiante	BASE TABLE
mai	estudiante	BASE TABLE
bda	estudiante	BASE TABLE

```

docencia => select schemaname, tablename
             from pg_catalog.pg_tables
             where schemaname in ('abd', 'bda', 'mai');

```

schemaname	tablename
abd	estudiante
mai	estudiante
bda	estudiante

A primeira consulta funcionaría en calquera sistema que utilizase o estándar `INFORMATION_SCHEMA` (por exemplo, MySQL/MariaDB ou versións recentes de Microsoft SQL Server).